VMSZC Horváth Boldizsár Közgazdasági és Informatikai Technikum

Az 5 0613 12 03 számú Szoftverfejlesztő- és tesztelő szakma

projektmunka

ABI

Raktárbérlés Online

Készítette:

Horváth Balázs

Szűcs Attila Márk

Katon András

SZOMBATHELY

2026

Tartalomjegyzék

Tartalomjegyzék.....2
1. Bevezetés.....3
2. A szoftver célja.....4
2.1. Felhasználói igények.....5
2.2. Funkcionális követelmények.....5
2.3. Funkcionális követelmények.....6
3. Komponensek technikai leírása.....7
3.1. Frontend.....7
3.2. Backend.....9
3.3. Adatbázis.....10
3.4. Verziókezelés.....14
3.5. Projektmenedzsment és feladatmegosztás.....16
4. Működés műszaki feltételei.....17
4.1. Hardveres követelmények.....18
4.2. Szoftveres követelmények.....19
4.2.1. Operációs rendszer.....19
4.2.2. Szükséges szoftverek.....20
4.2.3. Futtató környezetek (runtime).....20
4.2.4. Adatbáziskezelő.....21
4.2.5. Webszerver ismertetése.....21
4.3. Telepítés lépései.....21
5. Használat rövid bemutatása.....22
5.1. Regisztráció és felhasználói fiókok.....23
5.2. Főbb funkciók használata.....24
6. Monitorképek.....25
6.1. Bejelentkezés és Regisztráció.....25
6.2. Főoldal.....27
6.3. További képernyőképek.....29
7. Frontend tesztesetek:.....32
7.1. A tesztekhez végzett kód, valamint a teszteredmények dokumentációja:.....32
7.2. A tesztekhez végzett kód és a teszteredmények dokumentációja:.....33
7.3. Validációs egységtesztek – validators.js:.....33
7.4. Manuális funkcionális tesztelés:.....35
7.5. Tesztelési összefoglalás.....36

8. Backend tesztesetek.....37
8.1. A tesztekhez végzett kód, valamint a teszteredmények dokumentációja:.....37
8.2. A tesztelés megközelítése:.....37
8.3. API végpont tesztek összegzése:.....38
8.4. Negatív tesztesetek (hibás adatok kezelése):.....38
8.5. Futtatási eredmények:.....39
8.6. Tesztelési összefoglalás:.....39
9. Továbbfejlesztési lehetőségek.....39
Irodalomjegyzék.....40
1. melléklet – Adatbázis séma:.....41
2. melléklet – Projekt struktúra ER modell:.....42
3. melléklet – API dokumentáció:.....43

1. Bevezetés

A projektmunka célja egy olyan webes raktárbérlési rendszer tervezése és megvalósítása volt, amely egyszerre segíti az ügyfelek online ügyintézését és az adminisztrátorok napi munkáját. A választott témakör azért indokolt, mert a raktárbérlés iránt valós, folyamatos piaci igény van: egyre több magánszemély és kisvállalkozás keres rugalmas, digitálisan is könnyen kezelhető tárolási lehetőséget. A gyakorlatban sok esetben még mindig telefonos vagy e-mailes egyeztetéssel történik a bérlés, ami lassú, nehezen nyomon követhető és hibalehetőségekkel terhelt. A fejlesztett rendszer ezt a folyamatot digitalizálja.

A fejlesztés során egy többkomponensű, modern szoftveres megoldás készült: a felhasználói felületet frontend alkalmazás biztosítja, az üzleti logikát backend API szolgálja ki, az adatok pedig relációs adatbázisban tárolódnak. A rendszer támogatja többek között a regisztrációt, bejelentkezést, bérlések kezelését, e-mailes visszaigazolásokat és az ajtónyitáshoz kapcsolódó kódkezelést is.

A projektmunka a Szoftverfejlesztő szakma követelményeinek megfelelően készült. A dokumentáció felépítése és terjedelme az iskola által előírt formai és tartalmi követelményeket követi.

2. A szoftver célja

A szoftver elsődleges célja egy biztonságos, átlátható és felhasználóbarát raktárbérlési platform biztosítása, ahol az ügyfelek online tudnak tárolóhelyet választani, bérlést indítani és saját bérléseiket nyomon követni. A rendszer másik célja adminisztrátori oldalon az üzemeltetés támogatása: ügyfélkezelés, bérléskezelés, tároló- és raktáradatok karbantartása, valamint naplózható működés biztosítása.
A megoldani kívánt probléma az, hogy a hagyományos (telefonos, e-mailes, papíralapú) bérlési folyamat lassú, nehezen visszakövethető és sok kézi munkát igényel. A fejlesztett alkalmazás ezt a folyamatot digitalizálja, automatizálja és egységesíti.

A célközönség:

- magánszemélyek, akik ideiglenes tárolóhelyet keresnek,
- kis- és középvállalkozások, amelyek szezonális vagy tartós rakt

A RaktárBérlés egy modern, webes raktárbérlő rendszer, ami lehetővé teszi, hogy a felhasználók:

- Online válasszanak tárolót méret és helyszín alapján
- Lefoglalják a kívánt időszakra
- Távolról nyissák ki az ajtót egy 6 számjegyű kóddal
- Kezeljék a bérléseiket egy áttekinthető felületenározási igénnyel rendelkeznek,
- rendszergazdák/adminisztrátorok, akik a működést felügyelik.

A rendszer célja tehát nem csak egy egyszerű adatbeviteli felület, hanem egy teljes, kétoldalú megoldás: ügyféloldali használat + adminisztráció egy közös platformon, egy olyan alkalmazás , ami egyszerű, biztonságos és könnyen használható – akár egy idősebb felhasználó számára is.

2.1. Felhasználói igények

A feltárt felhasználói igények: - gyors és egyszerű regisztráció, - biztonságos azonosítás, - tárolóhelyek valós elérhetőségének megjelenítése, - online bérlési folyamat személyes ügyintézés nélkül, - megbízható visszajelzések (e-mail értesítések), - átlátható saját fiók és bérléskezelés, - adatbiztonság és jogosultság-alapú hozzáférés.

A tervezés során abból indultunk ki, hogy a felhasználók egyszerű és gyors felületet szeretnének, bonyolult lépések nélkül.

Ezért az alábbi igényekre koncentráltunk:

1. Egyszerű regisztráció és bejelentkezés.
2. Átlátható tárolólista, ahol látszik az elérhetőség.
3. Konkrét időszakra történő foglalás lehetősége.
4. Saját bérlések és profiladatok kezelése.
5. Biztonságosabb használat (jelszókezelés, jogosultságok).
6. Admin oldalon központi kezelés és ellenőrzés.

A rendszer ezekre úgy ad választ, hogy a frontend oldalon könnyen kezelhető menüpontok vannak, a háttérben pedig API-alapú működés végzi az adatok ellenőrzését és mentését. Így a felhasználó kevesebbet hibázik, és gyorsabban végez.

2.2. Funkcionális követelmények

Az alkalmazás fő funkciói az alábbiak:

1. Felhasználói fiókkezelés
   Regisztráció, bejelentkezés, email megerősítés, jelszó-visszaállítás, profilmódosítás.
2. Jogosultságkezelés
   Normál felhasználó és admin szerepkörök külön kezelése.

3. Tárolók listázása és szűrése

   A felhasználó meg tudja nézni a tárolóhelyiségeket és a hozzájuk tartozó adatokat.
4. Elérhetőség ellenőrzése dátum alapján

   Foglalás előtt a rendszer ellenőrzi, hogy az adott időszakban a tároló szabad-e.
5. Bérlés létrehozása

   A felhasználó új bérlést tud indítani, a rendszer számolja az összeget.
6. Fizetéshez kapcsolt foglalási folyamat

   A foglalás fizetési adatokkal is rögzíthető, számla jellegű visszaigazolással.
7. Saját bérlések kezelése

   A felhasználó látja a bérléseit és bizonyos esetekben törölni/lemondani is tudja.
8. Admin funkciók

   Megyék, raktárak, tárolók, árkategóriák, felhasználók és bérlések kezelése.
9. Email értesítések

   A rendszer különböző eseményeknél automatikusan emailt küld (pl. regisztráció, visszaállítás, foglalás).
10. Naplózhatóság és állapotkezelés

    A bérlések állapota és a fontosabb műveletek követhetők.

2.3. Funkcionális követelmények

- **Biztonság:** jelszóhash (BCrypt), JWT-alapú hozzáférésvédelem.
- **Megbízhatóság:** validációk, jogosultság-ellenőrzések, naplózott működés.
- **Karbantarthatóság:** rétegezett backend, külön service osztályok.
- **Bővíthetőség:** REST API szemlélet, moduláris komponensek.
- **Használhatóság:** egyszerű webes felület React alapon.
- **Adatkonzisztencia:** relációs adatmodell és EF Core leképezés.

3. Komponensek technikai leírása

Ez a fejezet a rendszer technikai felépítését mutatja be, külön fókuszálva a frontend, a backend és az adatbázis rétegre, valamint ezek együttműködésére. A megoldás egy többrétegű, webes architektúrát követ, amelyben a megjelenítés, az üzleti logika és az adatkezelés jól elkülönített felelősségi körökben működik. A rétegek közötti kommunikáció HTTP alapú REST API-n keresztül történik JSON adatformátummal, a hitelesítés pedig JWT tokennel valósul meg.

A teljes rendszer komponensei:

- Frontend kliensalkalmazás: React + Vite alapokon, böngészőben futó felhasználói felület.
- Backend API: ASP.NET Core Web API, üzleti logika, jogosultságkezelés, hitelesítés, e-mail folyamatok.
- Adatbázis réteg: MySQL, Entity Framework Core-on keresztül elérve.
- Kiegészítő szolgáltatások: SMTP e-mail küldés, PDF generálás, időzített háttérszolgáltatás.

A rendszer tudatosan úgy épül fel, hogy a kliens ne tartalmazzon üzleti döntési logikát, csak a felhasználói interakciók kezelését és az adatok megjelenítését. Az üzleti szabályok, validációk, jogosultsági vizsgálatok és kritikus folyamatok (például foglalás, státuszváltás, e-mail értesítések) a backend oldalon érvényesülnek. Ezzel biztosítható, hogy kliensoldali módosítás vagy manipuláció esetén se sérüljenek a működési szabályok.

3.1. Frontend

Alkalmazott technológiák és keretrendszerek:

React: 18.3.1:

Az alkalmazás felhasználói felülete a Facebook által fejlesztett React JavaScript könyvtár segítségével készült. A React komponens alapú architektúrát alkalmaz: az oldal egyes

részei (navigáció, lapok, kontextusok) önálló, újrafelhasználható komponensekként kerültek megvalósításra. A projekt kizárólag funkcionális komponenseket és React Hooks-ot használ (useState, useEffect, useContext, useRef, useCallback), class-alapú komponens nem szerepel a kódbázisban.

Vite: 5.4.10:

A fejlesztői környezet és a build folyamat alapja a Vite, amely natív ES modul importot és gyors HMR-t (Hot Module Replacement) biztosít. A hagyományos Create React App helyett Vite-ot alkalmaz a projekt, mivel lényegesen rövidebb indítási és újraépítési időket tesz lehetővé.

React Router DOM 6.27.0:

Az oldalak közötti navigációt a React Router v6 kezeli, kliens oldali útválasztással (SPA – Single Page Application). Az alkalmazás egyetlen HTML-fájlt tölt be (index.html), az oldalváltások JavaScript szinten történnek, nincs valódi lapújratöltés. A védett útvonalak (PrivateRoute, AdminRoute, GuestRoute) komponensek segítségével kerülnek lekorlátozásra.

HTML5:

Az alkalmazás belépési pontja az index.html fájl, amelybe a Vite injektálja a lefordított JavaScript modulokat. A komponensek JSX szintaxist alkalmaznak, amely a HTML-hez hasonló leírónyelvvel teszi lehetővé a felhasználói felület deklaratív felépítését.

CSS3 és Bootstrap 5.3.2:

A vizuális megjelenés alapja a Bootstrap 5, amely reszponzív rácsrendszert, előre elkészített UI-komponenseket (kártyák, accordion, modál, navigációs sáv, gomb) és segédosztályokat biztosít. A Bootstrap JavaScript funkcióit a projekt CDN-ről tölti be. Az egyedi stílusszabályok két külön fájlban találhatók:

- style.css – általános alkalmazásstílusok
- admin-style.css – adminisztrátori felület stílusai

JavaScript (ES2022+):

Az üzleti logika és az interaktív viselkedés modern ECMAScript szabvány szerint írt JavaScript kóddal valósul meg. A kód JSX formátumban (.jsx kiterjesztés) íródott, amelyet a Vite a Babel pluginjén keresztül fordít le böngésző-kompatibilis JavaScriptre.

Font Awesome 6.5.1:

Az ikonkészlet a Font Awesome CDN-ről kerül betöltésre. Az alkalmazás ikonjai (fa-warehouse, fa-lock, fa-key stb.) CSS osztályokon keresztül illeszthetők a komponensekbe.

Flatpickr 4.6.13:

A dátumválasztó vezérlő a Flatpickr könyvtár, amelyet a foglalási naptár megvalósításához használ az alkalmazás. Lehetővé teszi a bérlési időszak kezdő és záró dátumának kényelmes
kijelölését.

Kliens oldali állapotkezelés:

Az alkalmazás két React Context alapú állapotkezelőt alkalmaz:

- **AuthContext (src/context/AuthContext.jsx)** – a felhasználói munkamenet kezelése: JWT token tárolása localStorage-ban, token dekódolása (UTF-8 magyar karakterek támogatásával), bejelentkezési állapot és szerepkör közzététele az egész alkalmazás számára.
- **AlertContext (src/context/AlertContext.jsx)** – globális értesítési rendszer, amely kulcsszóalapú típusfelismeréssel (siker, figyelmeztetés, hiba) jelenít meg modális dialógusablakokat.

3.2. Backend

A rendszer szerveroldali része C# nyelven, ASP.NET Core Web API keretrendszerben készült. A backend feladata az üzleti logika kezelése, az adatok ellenőrzése, az API-végpontok biztosítása, valamint a jogosultságkezelés és a biztonsági folyamatok végrehajtása.

A backend főbb komponensei:

- kontrollerek: HTTP-kérések fogadása és válaszadás,
- service réteg: tokenkezelés, e-mailküldés, PDF generálás, háttérfolyamatok,
- adat-hozzáférési réteg: Entity Framework Core kapcsolódás az adatbázishoz,
- autentikációs és autorizációs modul: JWT és szerepköralapú hozzáférés.

Alkalmazott fontosabb technológiák:

- ASP.NET Core Web API,
- Entity Framework Core (ORM),
- JWT Bearer Authentication,
- BCrypt jelszó-hash,
- SMTP alapú e-mailküldés.

API-végpontok (főbb csoportok):

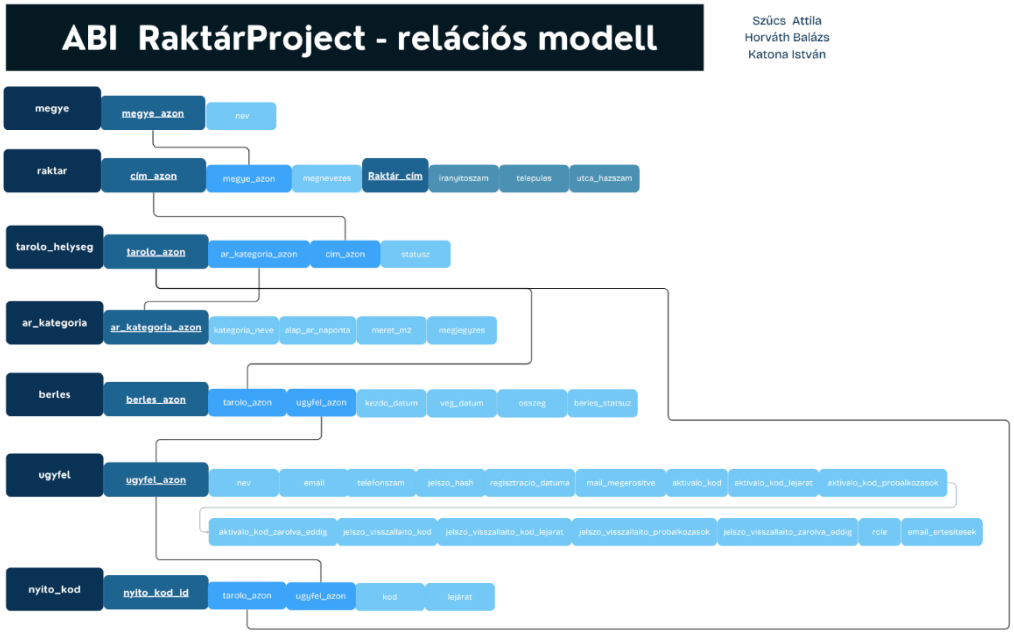
- Ugyfel: regisztráció, bejelentkezés, profilműveletek, e-mail verifikáció,
- Berles: bérlés létrehozás, listázás, állapotkezelés, kapcsolódó műveletek,
- TaroloHelyiseg: tárolóhelyek lekérdezése és adminisztrációja,
- Raktar és Megye: helyszín- és területi adatok kezelése,
- NyitoKod és Tavnyitas: nyitókódkezelés és távoli nyitási logika.

A backend kialakítása moduláris, így későbbi bővítéshez jól illeszthető (például új szolgáltatások vagy jogosultsági szintek hozzáadása esetén).

3.3. Adatbázis

A projekt relációs adatbázisra épül (MySQL/MariaDB), amely strukturáltan tárolja a felhasználók, raktárak, tárolóhelyek és bérlések adatait. Az adatmodell célja, hogy biztosítsa az adatok konzisztenciáját és a kapcsolatok egyértelmű kezelését.

Főbb entitások:



1.sz. kép: Relációs modell. Forrás: saját készítés.

- ugyfel: felhasználói adatok, jogosultságok, hitelesítési mezők,
- megye: területi besorolás,
- raktar: raktárhelyszínek adatai,
- ar_kategoria: ár- és méretkategóriák,
- tarolo_helyiseg: konkrét bérelhető egységek,
- berles: bérlési rekordok időintervallummal és státusszal,
- nyito_kod: időkorlátos hozzáférési kódok,
- nyitasi_log: ajtónyitási események naplója.

Kapcsolatok:

- egy megye több raktárat tartalmazhat (1:N),
- egy raktárhoz több tárolóhely tartozhat (1:N),
- egy felhasználónak több bérlése lehet (1:N),
- egy tárolóhelyhez időben több bérlési rekord kapcsolódhat (1:N),
- nyitókód és nyitási napló bérléshez/felhasználóhoz/tárolóhelyhez kötötten tárolódik.

A projektben az ORM szerepét az Entity Framework Core látja el, amely:

- leegyszerűsíti az adatbázis-műveleteket C# objektumokon keresztül,
- kezeli az entitáskapcsolatokat és navigációs tulajdonságokat,
- csökkenti a kézzel írt SQL mennyiségét,
- javítja a karbantarthatóságot és fejlesztési sebességet.

Összességében az adatbázis-struktúra a rendszer funkcionális céljaihoz jól illeszkedik, támogatja a napi működést és megfelelő alapot ad a későbbi bővítésekhez.

	Tábla	Művelet						Sorok	Típus	Illesztés	Méret	Felülírás
☐	ar_kategoria		Tartalom	Szerkezet	Keresés	Beszúrás	Kiürítés	Eldobás	2	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐	berles		Tartalom	Szerkezet	Keresés	Beszúrás	Kiürítés	Eldobás	3	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
☐	megye		Tartalom	Szerkezet	Keresés	Beszúrás	Kiürítés	Eldobás	3	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐	nyito_kod		Tartalom	Szerkezet	Keresés	Beszúrás	Kiürítés	Eldobás	0	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
☐	raktar		Tartalom	Szerkezet	Keresés	Beszúrás	Kiürítés	Eldobás	3	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐	tarolo_helyiseg		Tartalom	Szerkezet	Keresés	Beszúrás	Kiürítés	Eldobás	30	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
☐	ugyfel		Tartalom	Szerkezet	Keresés	Beszúrás	Kiürítés	Eldobás	2	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
	7 tábla	Összesen							43	InnoDB	utf8mb4_unicode_ci	272.0 KB	0 B

2.sz. kép: Adatbázis struktúra. Forrás: saját készítés.

JWT autentikáció:

A backend JWT Bearer hitelesítést használ. Bejelentkezés után a kliens tokent kap, amit minden védett API-hívásnál Authorization: Bearer <token> fejlécben küld.

A token validációja ellenőrzi: - aláírást, - kibocsátót (Issuer), - célközönséget (Audience), - lejáratot.

Jelszókezelés:

A jelszavak BCrypt algoritmussal hash-elve tárolódnak, nyers jelszó nem kerül mentésre.

Jogosultságkezelés:

A végpontok egy része szerepkörhöz kötött ([Authorize(Roles = "admin")]). Ez biztosítja, hogy admin műveleteket csak jogosult felhasználó végezhessen.

E-mail megerősítés és jelszó-visszaállítás:

- Regisztráció után e-mail megerősítési kód/alapú aktiválás történik.
- Elfelejtett jelszó esetén időkorlátos visszaállító folyamat működik.

A BCrypt algoritmus az egyik legjobb választás jelszavak tárolására, mert:

- Automatikusan generál salt-ot
- Lassú (szándékosan), így nehéz brute-force támadás
- Költségtényező állítható

```
// Regisztrációnál
string hash = BCrypt.Net.BCrypt.HashPassword("TitkosJelszo123");
// Eredmény: $2a$11$N9qo8uLOickgx2ZMRZoMy.random_salt_and_hash

// Bejelentkezésnél
bool helyes = BCrypt.Net.BCrypt.Verify("TitkosJelszo123", hash);
```

CORS:

A fejlesztői beállítás AllowAll policy. Ez fejlesztéshez kényelmes, de éles rendszerben szűkíteni kell konkrét domainekre.

Biztonsági megjegyzés:

Éles üzemhez kötelező: - titkok (JWT kulcs, SMTP jelszó) környezeti változóba helyezése, - naplózás és hibakezelés szigorítása, - CORS szűkítése, - rate limiting és brute-force védelem.

3.4. Verziókezelés

A projekt fejlesztése során Git verziókezelő rendszer alkalmazása javasolt és alkalmazott, amely lehetővé teszi a forráskód változásainak nyomon követését, a fejlesztési lépések dokumentálását és szükség esetén korábbi állapot visszaállítását.

A Git alapjai a projektben:

A Git egy elosztott verziókezelő rendszer, amely minden változtatást (commit) időbélyeggel, szerzővel és leírással tárol. A fejlesztés során minden jelentősebb funkcionalitás elkészültekor commit készült, így a fejlesztés menete visszakövethető.

A repository inicializálása az alábbi paranccsal történt:

git init - git add . - git commit -m "Kezdeti commit – projekt alapstruktúra"

Repositoryó struktúra:

A projekt két fő részből áll, amelyek egy közös repository-ban helyezkednek el:

RaktárProjekt

```
├── Frontend ←Reactfrontend
│ ├── src/
│ ├── package.json
│ └── vite.config.js
├── Backend ←.NETbackend
│ ├── RaktarProjekt.sln
│ └── RaktarProjekt/
│ ├── Controllers/
│ ├── Models/
│ ├── Service/
│ └── Program.cs
└── .gitignore
```

A .gitignore fájl gondoskodik arról, hogy a verziókezelőbe ne kerüljenek be felesleges vagy érzékeny fájlok, mint például:

node_modules/, dist/, bin/, obj/, appsettings.Development.json, *.user, .env

Branch-stratégia

A fejlesztés során az alábbi egyszerű branch-struktúra alkalmazható egy ilyen méretű projektben:

main – a stabil, működő verzió ága. Csak tesztelt, kész kód kerül ide.

dev – az aktív fejlesztési ág, ahol az új funkciók készülnek.

feature/xxx – egyedi funkciókhoz létrehozott ágak (pl. feature/nyitokod, feature/admin-panel), amelyek elkészülte után a dev ágba olvadnak vissza.

Jellemző commit üzenetek a projektből:

A fejlesztés lépései az alábbi típusú commitokkal dokumentálhatók:

- Kezdeti projekt struktúra, React + Vite beállítás
- Navbar és Footer komponens elkészítve
- AuthContext JWT token kezeléssel megvalósítva
- Regisztráció és email megerősítés funkció
- Tárolók böngészése és szűrés helyszín szerint
- Nyitókód generálás és mentés funkció
- Dashboard statisztikák megvalósítva
- Admin felület alapfunkciói elkészültek
- Validators.js form validáció hozzáadva
- Reszponzív stílusok javítása mobilon

Távoli repository (remote):

A kód biztonságos tárolásához és megosztásához GitHub vagy GitLab platform használható. Ez lehetővé teszi, hogy a kód online is elérhető legyen, és szükség esetén bármely eszközről visszaállítható legyen a legutóbbi állapot.

Összefoglalás:

A Git alapú verziókezelés alkalmazása a projekt fejlesztése során több szempontból is előnyös volt: lehetővé tette a fejlesztési lépések dokumentálását, a hibás változtatások visszavonását, valamint a frontend és backend kód egyidejű, strukturált kezelését. A

branch-stratégia alkalmazásával a stabil és a fejlesztési állapot egymástól elkülönítve kezelhető.

3.5. Projektmenedzsment és feladatmegosztás

A fejlesztési folyamat során a csapat agilis szemléletmódot alkalmazott, törekedve a szoros együttműködésre és a folyamatos tudásmegosztásra. Bár a tagok rendelkeztek elsődleges felelősségi körökkel, alapvető irányelv volt, hogy a projekt minden fázisára (tervezés, fejlesztés, tesztelés) minden csapattagnak teljes rálátása legyen, és aktívan részt vegyen a megoldások validálásában.

Együttműködési környezet és kommunikáció: A hatékony munkavégzés érdekében a csapat az alábbi infrastruktúrát és eszközöket használta:

- **Discord & Messenger:** A napi szintű operatív egyeztetések, gyors üzenetváltások és képernyőmegosztásos hibajavítások (pair programming) elsődleges helyszíne.
- **E-mail:** Hivatalosabb jellegű dokumentumok és adatok megosztására szolgált.
- **Google Drive:** A projekt-dokumentációk, specifikációk és egyéb nem kód jellegű állományok közös szerkesztésére és tárolására használtuk.
- **GitHub:** A központi verziókezelő rendszer, ahol a forráskódot tároltuk, és az ágkezelési stratégiák segítségével párhuzamosan tudtunk dolgozni.
- **Alkalmazott fejlesztői eszközök:** A különböző modulok fejlesztéséhez a csapattagok a feladathoz legoptimálisabb környezetet választották:
- **Visual Studio 2022 & 2026 (Preview):** Elsősorban a Backend és az API szolgáltatások fejlesztéséhez használtuk.
- **Visual Studio Code:** A Frontend fejlesztés, a React komponensek és a stíluslapok szerkesztésének fő eszköze volt.
- **Webböngészők (Chrome, Edge, Opera GX):** A kliensoldali felületek teszteléséhez és a fejlesztői konzol (DevTools) segítségével végzett debugoláshoz.
- **Munkamegosztás (Összesített nézet):** *Fontos megjegyezni, hogy az alábbi táblázat egy strukturált összefoglaló; a valós fejlesztési folyamat ennél jóval részletesebb és átfedőbb volt, mindenki részt vett minden fázis ellenőrzésében.*

Jelmagyarázat:

K – Készítette | E – Ellenőrizte

Feladatkör / Modul megnevezése	Szűcs Attila Márk	Horváth Balázs	Katona István
Frontend: Keretrendszer és architektúra	**K**	**E**	**K**
Frontend: Felhasználói és publikus UI	**K**	**E**	**K**
Frontend: Ügyfél bérlési folyamatok	**K**	**K**	**E**
Frontend: Adminisztrációs felületek	**E**	**K**	**E**
Backend: Rendszerfelépítés és Auth	**E**	**K**	**K**
Backend: Üzleti logika és API végpontok	**K**	**K**	**E**
Backend: Email és tranzakciós modulok	**K**	**E**	**K**
Adatbázis: Tervezés és sémaépítés	**K**	**K**	**E**
Integráció és E2E tesztelés	**K**	**K**	**K**
Dokumentáció és minőségbiztosítás	**K**	**K**	**K**

4. Működés műszaki feltételei

Ebben a fejezetben kerülnek ismertetésre azok a hardveres és szoftveres feltételek, amelyek szükségesek az alkalmazás biztonságos és stabil futtatásához. A rendszer két fő futtatási környezetből áll: egy szerver oldali backend komponensből és egy böngészőben futó frontend kliensből, amelyek eltérő, de egymást kiegészítő feltételeket igényelnek.

4.1. Hardveres követelmények

Az alkalmazás nem támaszt különleges hardveres igényeket, ugyanakkor a stabil és zavarmentes üzemeltetéshez érdemes figyelembe venni mind a minimális, mind az ajánlott konfigurációkat. Az alábbiakban mindkét szint külön kerül ismertetésre a szerver és a kliens oldal vonatkozásában.

Szerver (backend + adatbázis) – minimális konfiguráció:

Az ASP.NET Core backend és a MySQL adatbázis-szerver együttes futtatásához az alábbi minimális feltételek szükségesek:

- Processzor: legalább 2 magos, 64 bites CPU (x64 architektúra kötelező a .NET 8 futtatásához).
- Operatív memória (RAM): minimum 2 GB.
- Háttértároló: minimum 5 GB szabad lemezterület az alkalmazás binárisainak, az adatbázis fájljainak és a naplóknak.
- Hálózat: stabil internetkapcsolat, ha külső kliensek elérése szükséges.

Szerver (backend + adatbázis) – ajánlott konfiguráció:

A megbízható, hosszú távú üzemeltetéshez, különösen több egyidejű felhasználó esetén, az alábbi konfiguráció ajánlott:

- Processzor: 4 vagy több magos, 64 bites CPU, magasabb órajellel a gyorsabb

kérés-kiszolgáláshoz.
- Operatív memória (RAM): 8 GB vagy több, hogy az adatbázis-szerver és a backend párhuzamosan, megfelelő pufferrel fusson.
- Háttértároló: legalább 20 GB szabad lemezterület, lehetőleg SSD meghajtón a gyorsabb adatbázis-műveletek érdekében.
- Hálózat: legalább 10 Mbit/s szimmetrikus sávszélesség, alacsony késleltetéssel.

Kliens (böngésző, felhasználói oldal) – minimális konfiguráció:
A frontend egy böngészőben futó SPA alkalmazás, ezért kliensoldali telepítés nem szükséges. A minimális hardveres elvárások a böngésző általános igényeihez igazodnak:
- Processzor: bármely modern, kétmagos CPU.
- Operatív memória: minimum 2 GB RAM, amelyből legalább 512 MB elérhető a böngésző számára.
- Képernyő: minimum 360px szélességű megjelenítő (mobilon is működő reszponzív elrendezés).
- Hálózat: legalább 1 Mbit/s sávszélesség az API kommunikációhoz.

Kliens (böngésző, felhasználói oldal) – ajánlott konfiguráció:
A gördülékeny felhasználói élményhez, különösen az adminisztrációs felület és a táblázatos listák betöltésekor, az alábbi konfiguráció ajánlott:
- Processzor: négymagos vagy gyorsabb CPU.
- Operatív memória: 4 GB RAM vagy több.
- Képernyő: legalább 1280px szélességű megjelenítő az asztali nézet optimális megjelenítéséhez.
- Hálózat: legalább 5 Mbit/s sávszélesség a gyorsabb oldalbetöltéshez és API válaszidőkhöz.

4.2. Szoftveres követelmények

A fejezet az alkalmazás futtatásához szükséges szoftveres környezetet foglalja össze. A követelmények célja, hogy a rendszer telepítése, indítása és üzemszerű működése stabilan

biztosított legyen.

4.2.1. Operációs rendszer

A rendszer több operációs rendszeren is futtatható, de minden esetben 64 bites környezet szükséges.
- Támogatott Windows környezet: Windows 10, Windows 11, illetve Windows Server 2019 vagy újabb.
- Támogatott Linux környezet: Ubuntu 20.04 LTS vagy újabb, illetve más, modern disztribúciók .NET 8 támogatással.
- Fejlesztéshez és hibakereséshez Windows környezet ajánlott, mivel a projekt jelenlegi fejlesztési profiljai ezen lettek kialakítva.

4.2.2. Szükséges szoftverek

Az alkalmazás két fő részből áll, ezért a szükséges szoftverek is két csoportra oszthatók.

Backend oldal:
- ASP.NET Core alapú szerveralkalmazás.
- A futtatáshoz a .NET 8 platform szükséges.
- Fejlesztői fordításhoz és indításhoz .NET 8 SDK szükséges.

Frontend oldal:
- React alapú kliensalkalmazás.
- Fejlesztői környezetben Node.js 18 LTS vagy újabb és npm szükséges.
- A frontend futtatása végfelhasználói oldalon modern webböngészővel történik, különtelepítés nélkül.

4.2.3. Futtató környezetek (runtime)

A rendszer futtatása két runtime rétegben történik:

- Szerveroldali runtime: .NET 8 Runtime, amely a backend API végrehajtását biztosítja.
- Kliensoldali runtime: böngésző JavaScript motorja, amely a React alkalmazást futtatja.

Fejlesztési célra a runtime környezetekhez kapcsolódóan a következők szükségesek:
- .NET 8 SDK a backend fordításához és futtatásához.
- Node.js + npm a frontend fejlesztői build és fejlesztői szerver futtatásához.

4.2.4. Adatbáziskezelő

Az alkalmazás relációs adatbázist használ, amelyhez MySQL szerver szükséges:
- Elvárt verzió: MySQL 8.0 vagy újabb.
- Kapcsolódás: Entity Framework Core és MySQL provider segítségével.
- Üzemeltetési elvárás: állandó hálózati elérés a backend számára, valamint rendszeres mentés és naplózott működés.

Az adatbáziskezelő a felhasználói adatok, foglalási adatok és rendszerállapotok tartós tárolásáért felel, ezért a szolgáltatás folyamatos elérhetősége kritikus.

4.2.5. Webszerver ismertetése

A backend komponens ASP.NET Core webszerveren fut, amelynek alapértelmezett kiszolgálója a Kestrel.
- Fejlesztési környezetben a rendszer közvetlenül Kestrel alatt indul.
- A projekt beállításai alapján a fejlesztői elérés HTTP és HTTPS porton történik.
- Üzemi környezetben a Kestrel elé fordított proxy (például IIS vagy Nginx) helyezhető, amely kezeli a TLS lezárást, naplózást és forgalomirányítást.

A webszerver feladata az API végpontok kiszolgálása, a kérés-válasz ciklus kezelése, valamint a biztonságos hálózati kommunikáció biztosítása.

4.3. Telepítés lépései

A projekt telepítéséhez az alábbi előfeltételek szükségesek: Node.js (v18 vagy újabb) és npm csomagkezelő, valamint a háttérrendszer futtatásához .NET 7 SDK.

1. lépés – A forráskód letöltése:

A projekt könyvtárába lépés után klónozzuk vagy másoljuk a projektet:

cd \RaktarProjekt\raktar-react

2. lépés – Függőségek telepítése:

npm install

Ez telepíti az összes szükséges csomagot (React, React Router, Bootstrap, Flatpickr stb.) a package.json alapján.

3. lépés – Háttérrendszer indítása:

A React frontend egy .NET alapú REST API-val kommunikál, amely alapértelmezetten a *https://localhost:7195* címen fut. A Vite konfigurációjában proxy van beállítva, amely az */api/** kéréseket automatikusan erre a végpontra irányítja.

4. lépés – Fejlesztői szerver indítása:

npm run dev

Az alkalmazás a *http://localhost:5173* címen érhető el.

5. lépés – Éles build készítése (opcionális):

npm run build - Elkészíti a *dist/* mappát

npm run preview - Előnézet az éles buildből

5. Használat rövid bemutatása

A RaktárBérlés webalkalmazás raktárhelyiségek online bérlésére szolgál. A rendszert böngészőből, regisztráció után lehet teljes mértékben használni.

Az alkalmazás indítása után a főoldalon tájékozódhatunk az elérhető tárolókról, árakról és helyszínekről. A navigációs sáv tetején elérhető a *bejelentkezés/regisztráció*, a *kapcsolatfelvétel* és a *főoldal linkje*.

A felhasználói folyamat három lépésen alapul:

1. **Regisztráció** – A felhasználó megadja adatait (név, email, telefonszám, jelszó, születési dátum), majd e-mail-ben kapott 6 jegyű kóddal hitelesíti magát.
2. **Foglalás** – A Tárolók oldalon kiválasztja a helyszínt, a tároló típusát és a bérlési időszakot.
3. **Nyitás** – A bérlési idő alatt időkorlátos hozzáférési kódot kérhet a tároló kinyitásához, amely 10 percig érvényes.

5.1. Regisztráció és felhasználói fiókok

Ebben a részben a regisztrációs és bejelentkezési folyamat rövid, folyamatközpontú bemutatása található.

Regisztráció folyamata:

1. A felhasználó a Bejelentkezés oldalon a Regisztráció fülre vált.
2. Megadja az alapadatokat (név, e-mail cím, telefonszám, jelszó, születési dátum), majd elfogadja a vonatkozó feltételeket.
3. A kliensoldali űrlapellenőrzés lefut (kötelező mezők, formátumok, jelszóerősség, életkor-ellenőrzés).
4. Sikeres ellenőrzés után a rendszer létrehozza a fiókot, majd aktiváló kódot küld a megadott e-mail címre.
5. A felhasználó az aktiváló kód megadásával megerősíti az e-mail címét.
6. Amennyiben a kód lejár vagy nem érkezik meg, a rendszer lehetőséget ad az aktiváló kód újraküldésére.

Fontos működési szabály, hogy megerősítés nélkül a fiók ugyan létrejön, de bejelentkezésre még nem jogosult.

Bejelentkezés folyamata:

1. A felhasználó e-mail címmel és jelszóval bejelentkezést kezdeményez.
2. A backend ellenőrzi, hogy a felhasználó létezik-e, a jelszó helyes-e, valamint

megtörtént-e az e-mail megerősítés.

3. Sikeres bejelentkezés esetén a rendszer JWT tokent ad vissza.

4. A kliens eltárolja a tokent, majd a felhasználó szerepköre alapján a megfelelő felületre navigál (admin vagy felhasználói irányítópult).

Hibakezelés és fiókbiztonság röviden:

- Hibás hitelesítési adatok esetén a rendszer nem engedi be a felhasználót, és egyértelmű visszajelzést ad.
- Nem megerősített e-mail címnél a rendszer aktiválási folyamatra tereli vissza a felhasználót.
- Elfelejtett jelszó esetén külön visszaállítási folyamat érhető el: kódkérés e-mailben, majd a kód megerősítése és új jelszó beállítása.
- A sikeres hitelesítés után kiadott token biztosítja a jogosultságalapú hozzáférést.

Összességében a fiókkezelési folyamat célja, hogy a regisztráció egyszerű maradjon, miközben a belépés biztonsági ellenőrzései (jelszó, e-mail megerősítés, tokenes azonosítás) megfelelő védelmet adjanak.

5.2. Főbb funkciók használata

Regisztráció és bejelentkezés:

A */bejelentkezes* oldalon érhető el mind a belépés, mind a regisztráció. Regisztráció után az e-mail-cím hitelesítése kötelező. Elfelejtett jelszó esetén e-mailes visszaállítás érhető el.

Tárolók böngészése és foglalás:

A */tarolok* oldalon a felhasználó helyszín és típus szerint szűrheti az elérhető raktárhelyiségeket.

Kétféle típus közül választhat:

Basic (500 Ft/nap) – száraz, zárt tároló, saját lakat, világítás

Premium (600 Ft/nap) – klímavezérelt, polcos, jobb megvilágítás

A foglaláshoz egyedi dátumtartomány adható meg a Flatpickr naptárvezérlővel.

Hozzáférési kód generálás:

Aktív bérlés esetén a */berleseim* oldalon ideiglenes nyitókód igényelhető, amely pontosan 10 percig érvényes – fizikai kulcs nem szükséges.

Dashboard:

A */dashboard* oldalon áttekinthető az aktív, függőben lévő és lejárt bérlések száma, az 5 legutóbbi bérlés és a teljes kiadás összesítve.

Profil szerkesztése:

A */profil* oldalon a felhasználó módosíthatja személyes adatait (megjelenítési név, jelszó).

Admin felület:

Az */admin* útvonalon – kizárólag adminisztrátor szerepkörű felhasználóknak – elérhető a tárolók és bérlések kezelése.

Fizetés:

A */fizetes* oldalon kezeli a rendszer a bérlési díjak kiegyenlítését.

6. Monitorképek

Ebben a fejezetben kerülnek bemutatásra az alkalmazás legfontosabb képernyőképei, amelyek szemléltetik a felhasználói felületet és az egyes funkciók megjelenését.

6.1. Bejelentkezés és Regisztráció

A bejelentkezési felület a felhasználó elsődleges belépési pontja, ahol azonosított hozzáférés történik az alkalmazás szolgáltatásaihoz.

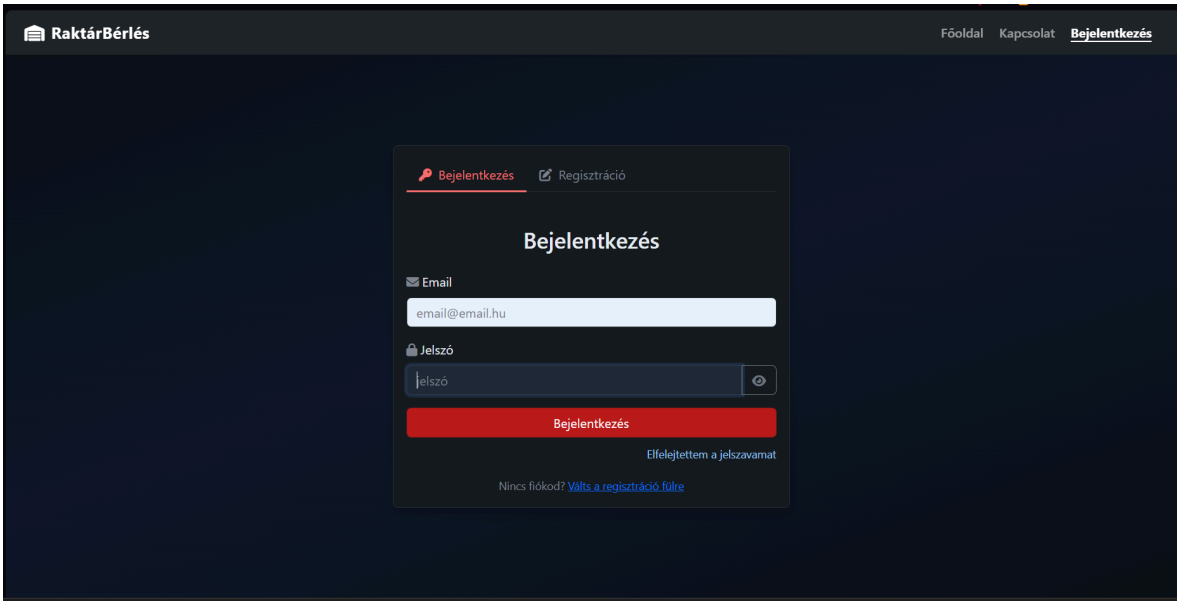
Bejeletnkezés rövid működési leírás:

1. A felhasználó megadja e-mail címét és jelszavát.
2. A rendszer ellenőrzi a hitelesítő adatokat és az e-mail megerősítés állapotát.
3. Sikeres belépés esetén jogosultságalapú továbbirányítás történik.
4. Sikertelen belépésnél a felület célzott visszajelzést ad (hibás jelszó, nem létező

e-mail, nem megerősített fiók).

5. A bejelentkezési nézeten keresztül érhető el a regisztráció és a jelszó-visszaállítás is, ezért ez a képernyő a teljes felhasználói fiókéletciklus központi kiindulópontja.

Bejelentkezés:

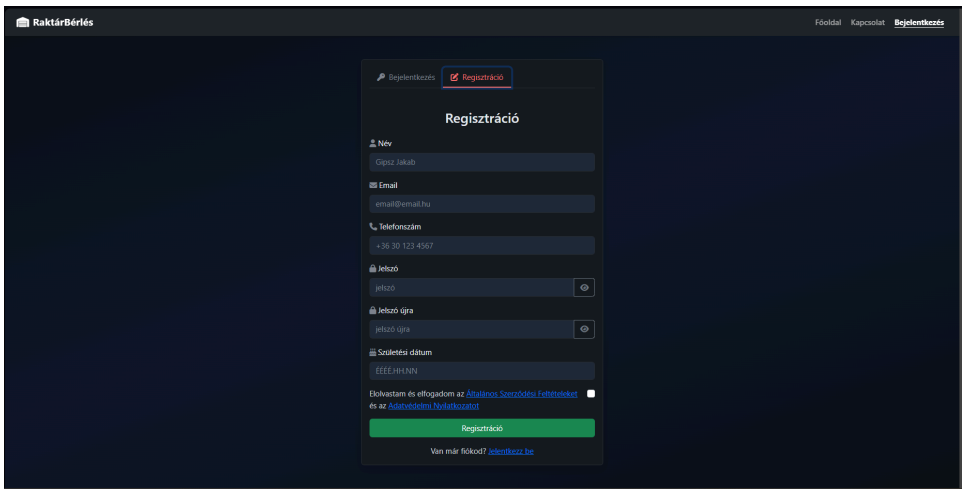


3.sz. kép: Bejelentkezési felület – Bejelentkezési rész. Forrás: saját készítés.

Regisztráció rövid működési leírás:

1. Az új felhasználó megadja a szükséges adatokat (név, e-mail cím, jelszó, opcionálisan telefonszám).
2. A rendszer ellenőrzi a kötelező mezőket, az e-mail formátumát és az e-mail cím egyediségét.
3. Sikeres adatellenőrzés után a rendszer létrehozza a fiókot, és megerősítő e-mailt küld aktiváló kóddal vagy aktivációs hivatkozással.
4. Aktiválás nélkül a fiók nem használható teljeskörűen, így a rendszer védi a felhasználót a hibás vagy illetéktelen regisztrációktól.
5. A regisztrációs hibákról a felület célzott visszajelzést ad (már létező e-mail, hiányzó mező, érvénytelen formátum).

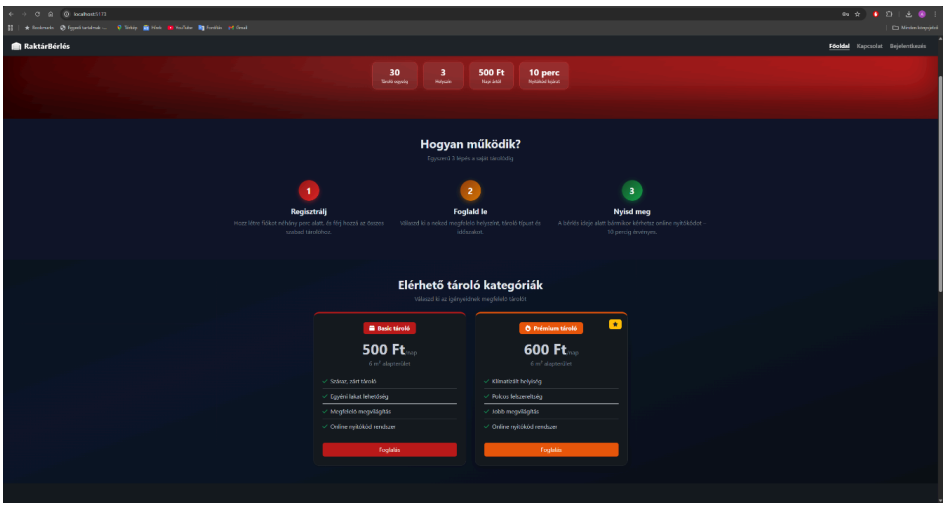
Regisztráció:



4. sz. kép: Bejelentkezési felület - Regisztráció rész. Forrás: saját készítés.

6.2. Főoldal

A főoldal (*src/pages/Home.jsx*) az alkalmazás nyitólapja, amely publikusan, bejelentkezés nélkül is elérhető. Feladata a szolgáltatás bemutatása és az érdeklődők tájékoztatása.

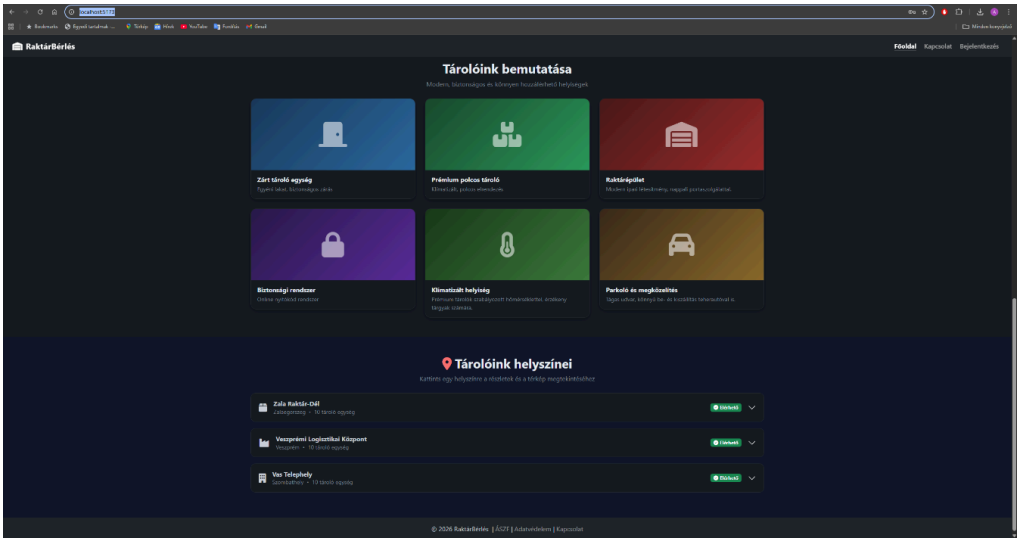


5. sz. kép: Főoldal. Forrás: saját készítés.

Hero szekció – Az oldal tetején animált statisztikák jelennek meg (30 tároló, 3 helyszín, 500 Ft/nap kezdőár, 10 perces kódérvényesség), amelyek görgetés hatására számlálóanimációval töltődnek be. A fő szlogen: „Raktárhelyiségek bérlése egyszerűen".

Hogyan működik? – Három ikonos lépésben mutatja be a bérlési folyamatot (Regisztráció → Foglalás → Nyitás).

Tárolótípusok – Kártya formátumban jeleníti meg a Basic és Premium csomagokat, az előnyök listájával és az árakkal. A Premium csomag „Ajánlott" jelöléssel van kiemelve.



6. sz. kép: Főoldal. Forrás: saját készítés.

Galéria / Bemutató – Hat kártya szemlélteti a raktár különböző aspektusait (zárt egységek, polcos tárolók, épület, kamerarendszer, klímás helyiség, parkoló). Kártyára kattintva modális ablak nyílik részletes leírással, előző/következő navigációval és billentyűzetes vezérléssel (Escape, nyilak).

Helyszínek – Bootstrap összecsukható (accordion) komponensben jelenik meg a három helyszín:

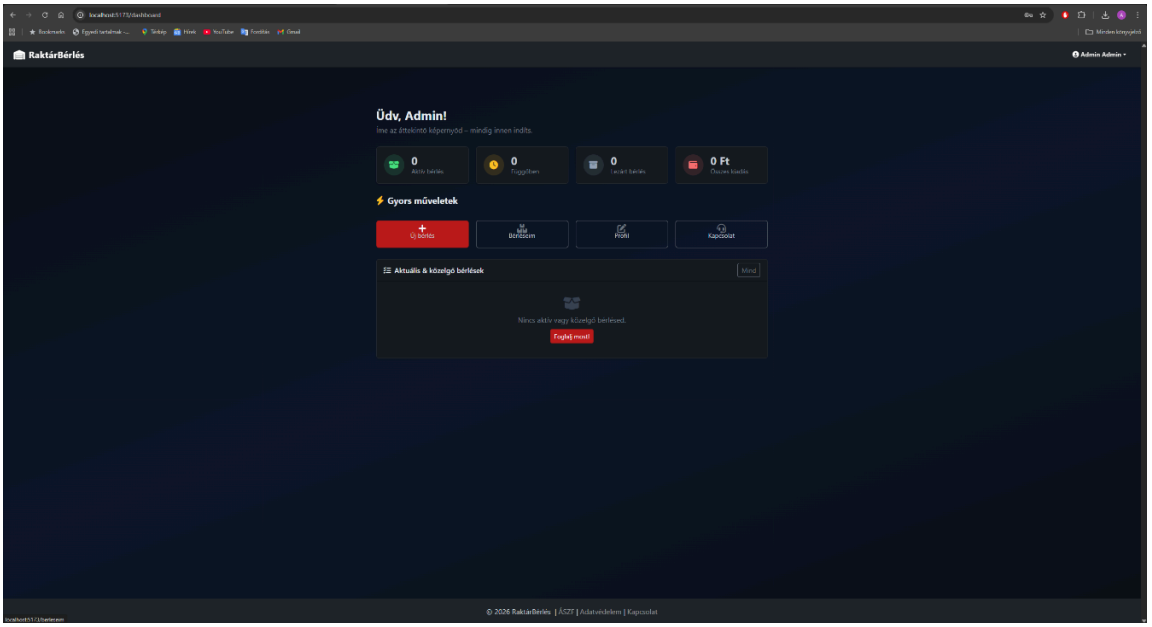
- Zala Raktár-Dél – Zalaegerszeg, Ipari Park utca 4.

- Veszprémi Logisztikai Központ – Veszprém, Logisztikai Centrum út 3.
- Vas Telephely – Szombathely, Hűtőház utca 5.

Minden helyszínnél megjelenik az elérhető tárolók száma, a beágyazott Google Maps térkép és a Google Maps hivatkozás.

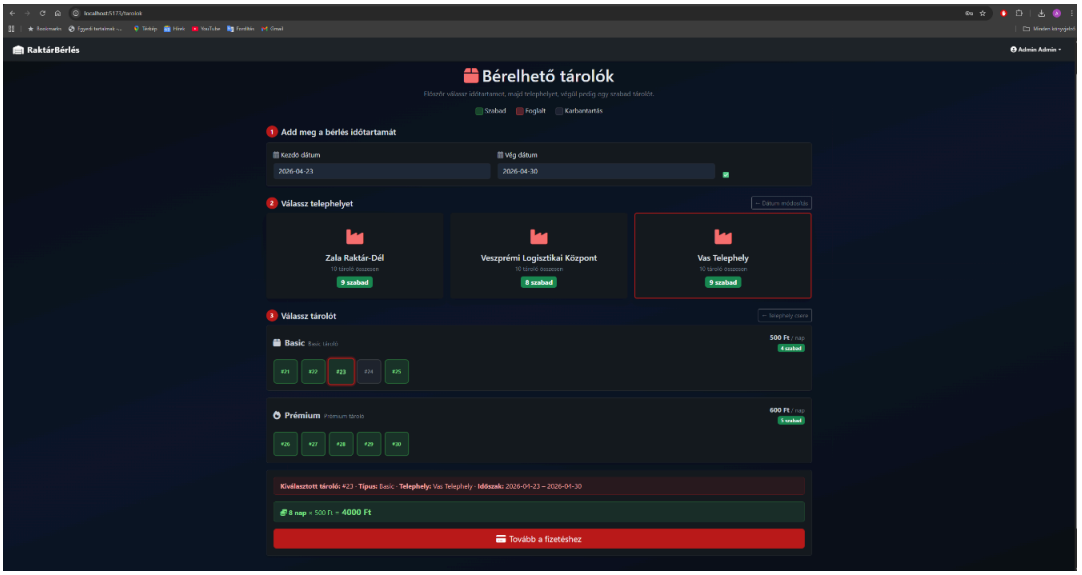
6.3. További képernyőképek

Dashboard:



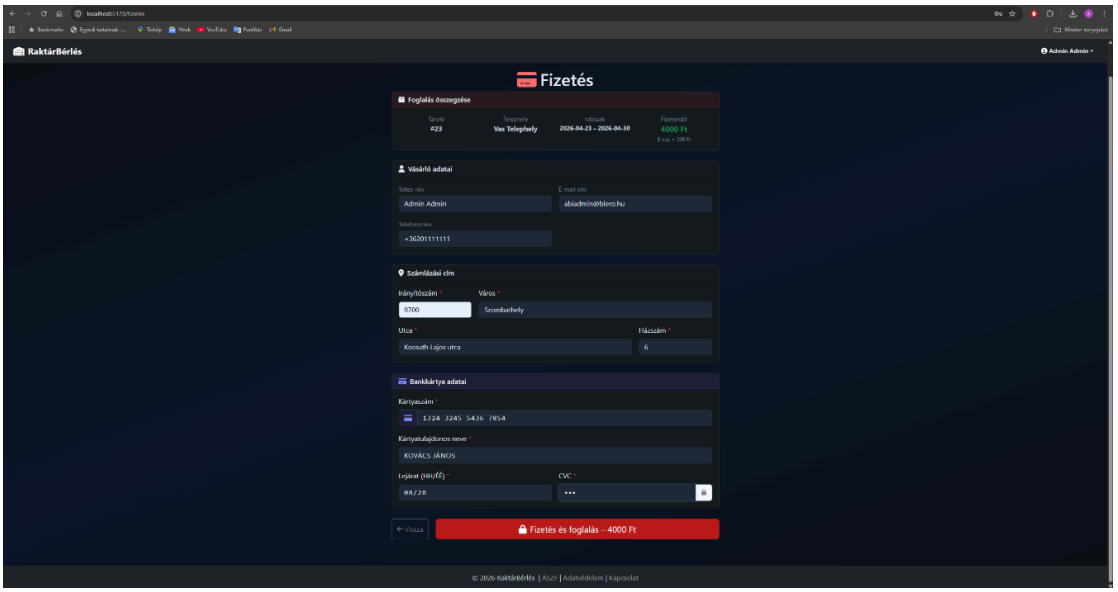
7. sz. kép: Deshbord. Forrás: saját készítés.

Fizetés és bérlés előtt:



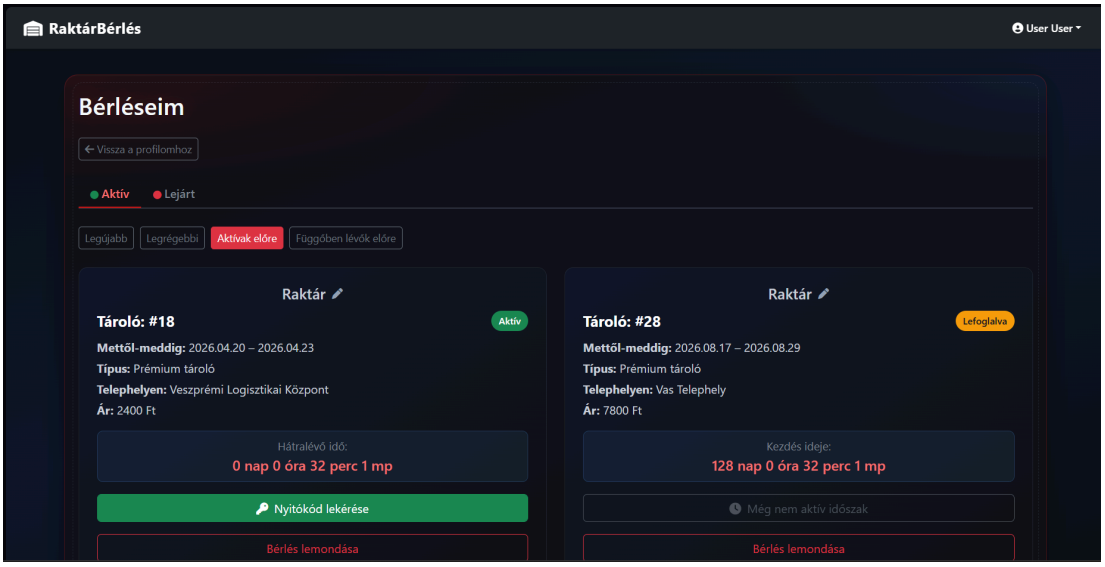
8. sz. kép: Bérlés oldal. Forrás: saját készítés.

Fizetés:



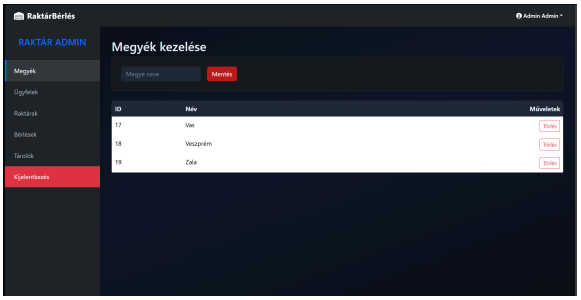
9. sz. kép: Fizetés oldal. Forrás: saját készítés.

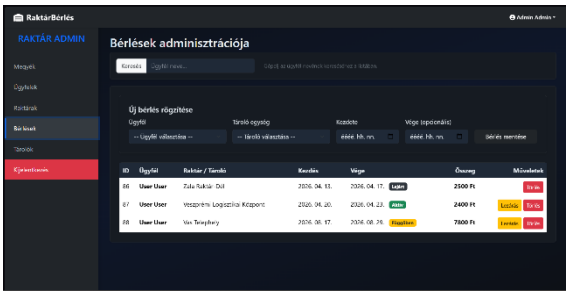
Fizetés után a Bérléseim oldalon:



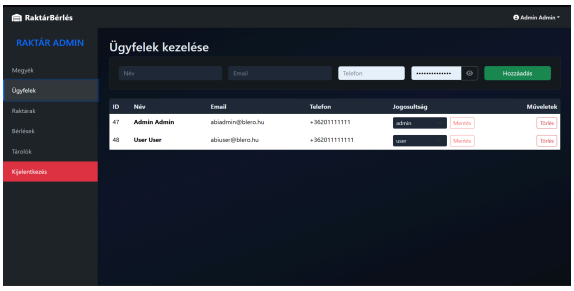
10. sz. kép: Bérléseim oldal. Forrás: saját készítés.

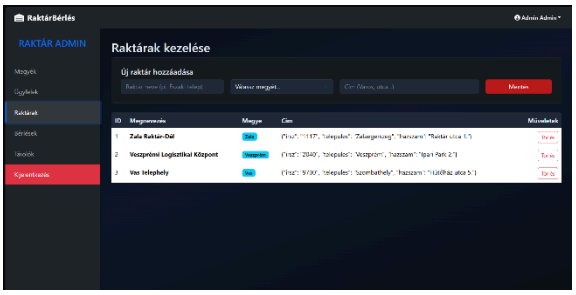
Admin oldal:



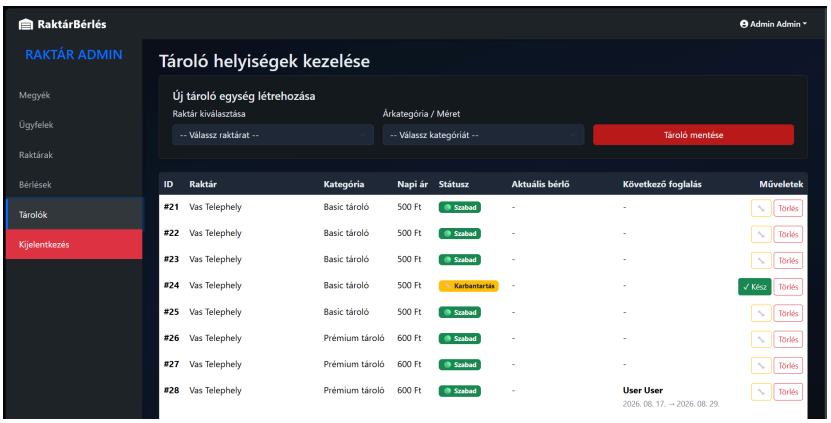


11-12. sz. kép: Admin oldal – Megyék-Bérlések lap. Forrás: saját készítés.





13-14. *sz. kép: Admin oldal- Ügyfelek-Raktárak lap. Forrás: saját készítés.*



15. *sz. kép: Admin oldal - Tárolók lap. Forrás: saját készítés.*

7. Frontend tesztesetek:

7.1. A tesztekhez végzett kód, valamint a teszteredmények dokumentációja:

Ebben a fejezetben rövid bemutatásra kerül a projektmunka célja, a választott témakör indoklása, valamint az elkészített szoftver általános ismertetése. A projektmunka a **Szoftverfejlesztő és -tesztelő** szakma követelményeinek megfelelően készült.

A projekt egy **raktárhelyiség-bérléssel foglalkozó webalkalmazás** – neve: *RaktárBérlés* –, amelynek célja, hogy a felhasználók egyszerűen, interneten keresztül foglalhassák és kezelhessék a bérelt raktáraikat, fizikai jelenlét és papíralapú ügyintézés nélkül. A témaválasztást az indokolta, hogy a raktárbérlési piac egyre nagyobb igényt támaszt az online, önkiszolgáló megoldások iránt, ugyanakkor kevés hazai platform kínál ilyen jellegű, felhasználóbarát digitális rendszert.

Az elkészített szoftver egy modern, böngészőalapú SPA (Single Page Application), amely React keretrendszerrel épül fel, és egy .NET háttérrendszerrel kommunikál REST API-n keresztül. A rendszer támogatja a regisztrációt, a bejelentkezést, a tárolók böngészését és foglalását, az online hozzáférési kód generálást, valamint adminisztrációs feladatok elvégzését.

7.2. A tesztekhez végzett kód és a teszteredmények dokumentációja:

A projekt fejlesztése során automatizált tesztkeretrendszer (pl. Vitest, Jest) nem került bevezetésre – a tesztelés **manuális funkcionális teszteléssel** valósult meg böngészőkörnyezetben. Ugyanakkor az alkalmazás logikájának legkritikusabb, izolálható egységei – az *src/utils/validators.js* fájlban található validációs függvények – önállóan, kézzel is ellenőrizhetők és tesztelhetők, mivel tiszta, mellékhatás-mentes függvények.

7.3. Validációs egységtesztek – validators.js:

Az alábbi táblázatok az egyes validációs függvények teszteseteit és elvárt eredményeit tartalmazzák.

checkName(name) – Teljes név ellenőrzése:

#	Bemeneti érték	Elvárt eredmény	Magyarázat
1	""	isEmpty: true	Üres mező
2	"kovács béla"	isFormat: true	Nincs nagybetűs kezdés
3	"Kovács"	isFormat: true	Hiányzik a keresztnév
4	"Kovács Béla"	isEmpty: false, isFormat: false	Helyes formátum
5	"Ős Árpád"	isEmpty: false, isFormat: false	Magyar ékezetes nagybetű

checkEmail(email) – E-mail cím ellenőrzése:

#	Bemeneti érték	Elvárt eredmény	Magyarázat
1	""	isEmpty: true	Üres mező
2	"nememail"	isFormat: true	Nincs @ jel
3	"a@b"	isFormat: true	Hiányzik a domain kiterjesztés
4	"teszt@gmail.com"	isEmpty: false, isFormat: false	Helyes formátum
5	"teszt.nev+tag@domain.hu"	isEmpty: false, isFormat: false	Összetett, érvényes cím

checkPhone(phone) – Telefonszám ellenőrzése:

#	Bemeneti érték	Elvárt eredmény	Magyarázat
1	""	isEmpty: true	Üres mező
2	"abc"	isFormat: true	Nem számjegy
3	"123"	isFormat: true	Túl kevés számjegy (min. 7)
4	"+36 30 123 4567"	isEmpty: false, isFormat: false	Magyar formátum, helyes
5	"(06) 1 234-5678"	isEmpty: false, isFormat: false	Alternatív helyes formátum

checkPassword(password) – Jelszó erősség ellenőrzése:

#	Bemeneti érték	Elvárt eredmény	Magyarázat
1	"abc"	tooShort: true	Kevesebb mint 8 karakter
2	"abcd1234"	noUpper: true, noSpecial: true	Nincs nagybetű, nincs spec. karakter
3	"Abcd1234"	noSpecial: true	Hiányzik a speciális karakter
4	"Abcd123!"	minden false	Minden feltétel teljesül
5	"Jelszo1! "	forbiddenChar: true	Szóköz tiltott karakter

parseBirthday(value) és isOver18FromDate() – Születési dátum és korhatár:

#	Bemeneti érték	Elvárt eredmény	Magyarázat
1	""	null	Üres mező
2	"2000.13.01"	null	Érvénytelen hónap
3	"2000.02.30"	null	Nem létező nap
4	"2000.06.15"	Date objektum	Érvényes dátum
5	isOver18(születési dátum 2010-ből)	false	Nem töltötte be a 18. évet
6	isOver18(születési dátum	true	Betöltött 18 év

#	Bemeneti érték	Elvárt eredmény	Magyarázat
	2000-ből)		

7.4. Manuális funkcionális tesztelés:

Az alkalmazás teljes körű tesztelése böngészőben (Google Chrome) történt, az alábbi fő folyamatokra kiterjedően:

Regisztráció és bejelentkezés:

#	Teszt leírása	Elvárt viselkedés	Eredmény
1	Regisztráció hiányos adatokkal	Hibaüzenetek megjelenése az érintett mezőknél	✓ **Megfelel**
2	Regisztráció érvényes adatokkal	E-mailes visszaigazolás, kód bekérése	✓ **Megfelel**
3	Hibás 6 jegyű kód megadása	Hibaüzenet, újrapróbálkozás lehetősége	✓ **Megfelel**
4	Bejelentkezés rossz jelszóval	Hibaüzenet megjelenése	✓ **Megfelel**
5	Bejelentkezés helyes adatokkal	Átirányítás a főoldalra, navbar változása	✓ **Megfelel**

Tárolók böngészése és foglalás:

#	Teszt leírása	Elvárt viselkedés	Eredmény
6	Tároló oldal megnyitása bejelentkezés nélkül	Átirányítás a bejelentkezési oldalra	✓ **Megfelel**
7	Tárolók szűrése helyszín szerint	Csak a kiválasztott helyszín tárolói jelennek meg	✓ **Megfelel**
8	Foglalás érvényes dátumtartománnyal	Sikeres foglalás visszaigazolása	✓ **Megfelel**
9	Foglalás múltbeli dátummal	A naptár nem engedi kiválasztani	✓ **Megfelel**

Hozzáférési kód és bérlések kezelése:

#	Teszt leírása	Elvárt viselkedés	Eredmény
10	Hozzáférési kód igénylése aktív bérlésnél	10 perces érvényességű kód megjelenik	✓ **Megfelel**
11	Lejárt bérlés megjelenítése	„Lejárt" állapot látható a bérlések listájában	✓ **Megfelel**

Admin felület:

#	Teszt leírása	Elvárt viselkedés	Eredmény
12	Admin oldal elérése normál felhasználóként	Átirányítás a főoldalra	✓ **Megfelel**
13	Admin oldal elérése admin felhasználóként	Az adminisztrációs felület betöltődik	✓ **Megfelel**

Reszponzivitás:

#	Teszt leírása	Elvárt viselkedés	Eredmény
14	Mobilnézet (375px szélesség)	A navigáció hamburger menüvé alakul, a tartalom egymás alá kerül	✓ **Megfelel**
15	Tabletnézet (768px)	Kétoszlopos elrendezés, olvasható szövegek	✓ **Megfelel**

7.5. Tesztelési összefoglalás

Az elvégzett tesztek alapján az alkalmazás az összes vizsgált funkcióban az elvárásoknak megfelelően működik. Az inputvalidáció megbízhatóan kiszűri a hibás adatokat, az útvonalvédelem helyesen kezeli a jogosultságokat, a felhasználói felület reszponzív és mobilbarát. Automatizált tesztek jövőbeli bevezetése (pl. **Vitest** egységtesztekhez, **Playwright** végponttesztekhez) tovább növelné a kódbázis megbízhatóságát.

8. Backend tesztesetek

8.1. A tesztekhez végzett kód, valamint a teszteredmények dokumentációja:

Ebben a fejezetben bemutatásra kerül a backend oldali tesztelés célja, a választott tesztelési stratégia, valamint az elvégzett tesztek eredménye. A tesztek a .NET alapokon épülő REST API vezérlők működésének ellenőrzésére készültek, külön hangsúllyal a hibás bemenetekre adott válaszokra és a státuszkódok helyes kezelésére.

A backend célja, hogy megbízható, biztonságos és kiszámítható API viselkedést biztosítson a frontend alkalmazás számára. A tesztelés során ellenőrzésre került a CRUD műveletek helyessége, a jogosultsághoz kötött ágak viselkedése, valamint a validációs és hibaágak visszatérési értékei.

8.2. A tesztelés megközelítése:

A projektben automatizált backend tesztek készültek xUnit keretrendszerrel. Az adatbázis-függő logikák validálásához EF Core InMemory providert használtunk, így a tesztek külső MySQL kapcsolat nélkül, gyorsan és reprodukálható módon futtathatók.

Elem	Alkalmazott megoldás	Indoklás	Eredmény
Teszt típusa	Controller szintű unit tesztek	Gyors futás, izolált logikai ellenőrzés	Stabil, reprodukálható
Teszt keretrendszer	xUnit (.NET 8)	Széles körben használt .NET teszt keretrendszer	Integrált, CI-kompatibilis
Adatbázis	EF Core InMemory	Külső MySQL-függés nélküli futtatás	Gyors, lokálisan futtatható
Hatókör	CRUD + validációs + jogosultsági ágak	Funkcionális kimenetek ellenőrzése	23/23 sikeres

8.3. API végpont tesztek összegzése:

A tesztek controller szinten ellenőrzik az ActionResult kimeneteket, a visszaküldött objektummezőket, valamint a hibás bemenetekre adott megfelelő HTTP válaszokat.

Kategória	Tesztek száma	Sikeres	Lefedett terület
ÁrKategória, Megye, Raktár	10	10	CRUD + 400/404 hibaágak
Tárolóhelyiség, Nyitókód	10	10	Státuszlogika + kódérvényesség + 401/403
Bérlés (elérhetőség)	3	3	Dátumvalidáció + foglalt/szabad lista
Összesen	23	23	0 hiba, 0 skip

8.4. Negatív tesztesetek (hibás adatok kezelése):

A backend egyik legfontosabb minőségi követelménye, hogy hibás bemenet esetén is egyértelmű, konzisztens választ adjon. Az alábbi esetekben ezt külön ellenőriztük.

#	Teszteset	Hibás bemenet vagy állapot	Kapott rendszerreakció
1	ArKategoria_GetById_Missing	Nem létező azonosító	404 NotFound + hibaobjektum
2	ArKategoria_Put_MismatchedId	URL ID != body ID	400 BadRequest
3	Raktar_Delete_Missing	Nem létező rekord törlése	404 NotFound
4	NyitoKod_Ellenorzes_InvalidInput	tarolo <= 0 vagy kód üres	403 tiltott kérés jellegű válasz
5	NyitoKod_Ellenorzes_ExpiredCode	Lejárt kód	403 + törlés az adatbázisból
6	NyitoKod_DeleteNyitoKod_MissingClaim	Hiányzó NameIdentifier claim	401 Unauthorized
7	Berles_Elerhetoseg_MinValueDates	Hiányzó érvényes dátum	400 BadRequest
8	Berles_Elerhetoseg_StartAfterEnd	Kezdődátum > végdátum	400 BadRequest

8.5. Futtatási eredmények:

Mutató	Érték
Futtatási parancs	dotnet test RaktarProjekt.sln
Összes teszt	23
Sikeres	23
Hibás	0
Kihagyott	0
Időtartam	kb. 1,4s - 3,5s

8.6. Tesztelési összefoglalás:

Az elvégzett backend tesztek alapján az API a vizsgált területeken megfelelően működik. A validációs és hibaágak helyes státuszkódokat adnak vissza, az entitásműveletek konzisztensek, és a kritikus vezérlőági logikák megbízhatóan viselkednek.

A jövőbeli fejlesztések során javasolt HTTP integrációs tesztek és e2e tesztek bevezetése (például WebApplicationFactory, Playwright API-szint), hogy a model binding típushibák (például szám helyett szöveg JSON-ban) és a teljes request pipeline viselkedése is automatikusan ellenőrizhető legyen.

9. Továbbfejlesztési lehetőségek

Az alkalmazás jelenlegi állapotában egy stabil MVP-t képvisel, amelyet számos irányban lehet bővíteni:

Funkcionális bővítések:

- Online fizetési átjáró integrálása (pl. Stripe, Barion, SimplePay) – jelenleg a fizetési oldal kiépítés alatt van.
- Valós idejű értesítések – WebSocket vagy push notification alapú rendszer a bérlés lejártának és a kódok érvényességének jelzéséhez.
- Naptáras foglalási nézet – vizuális, drag-and-drop-os naptár a szabad időszakok gyorsabb áttekintéséhez.
- QR-kódos beléptetés – az ideiglenes hozzáférési kód QR-kódként is megjeleníthető lenne mobileszközökön.
- Értékelési rendszer – bérlők értékelhetik a helyszíneket bérlés után.

Technikai fejlesztések:

- Tesztek bevezetése – egység- és integrációs tesztek (pl. Vitest, React Testing Library) a stabilitás növelésére.
- Állapotkezelés centralizálása – nagyobb léptékű fejlesztés esetén Redux Toolkit vagy Zustand bevezetése.
- PWA (Progressive Web App) támogatás – offline működés és telefonra telepíthetőség.
- Teljesítményoptimalizálás – lusta betöltés (lazy loading) a route-ok szintjén, képoptimalizálás.
- Többnyelvűség (i18n) – angol és egyéb nyelvi változat hozzáadása (pl. react-i18next).

Adminisztrációs fejlesztések:

- Riport és statisztika modul – bevételi kimutatások, foglaltsági grafikonok az admin felületen.
- Automatikus e-mail értesítők – bérlés lejárata előtt emlékeztető küldése.
- Karbantartási naptár – tárolók karbantartás miatti blokkolása az adminisztrátori felületről.

Irodalomjegyzék

A hivatkozások APA formátumban kerülnek feltüntetésre.

ELTE Informatikai Kar. (2026.03.08). *JavaScript és webes technológiák*. https://inf.elte.hu

Progmasters. (2026.04.16). *React fejlesztői útmutató magyarul*. https://progmasters.hu

Webfejlesztés.hu. (2026.04.18). *Modern JavaScript – ES6 és újabb szabványok*. https://webfejlesztes.hus

W3Schools Magyar. (2025.12.13). *HTML5, CSS3 és JavaScript oktatóanyag*. https://w3schools.com

DevCentral Hungary. (2026.03.29). *ASP.NET Core REST API fejlesztés*.

https://devcentral.hu

HWSW Informatikai Hírmagazin. (2026.02.09). *Szoftverfejlesztési módszertanok és eszközök*. https://hwsw.hu

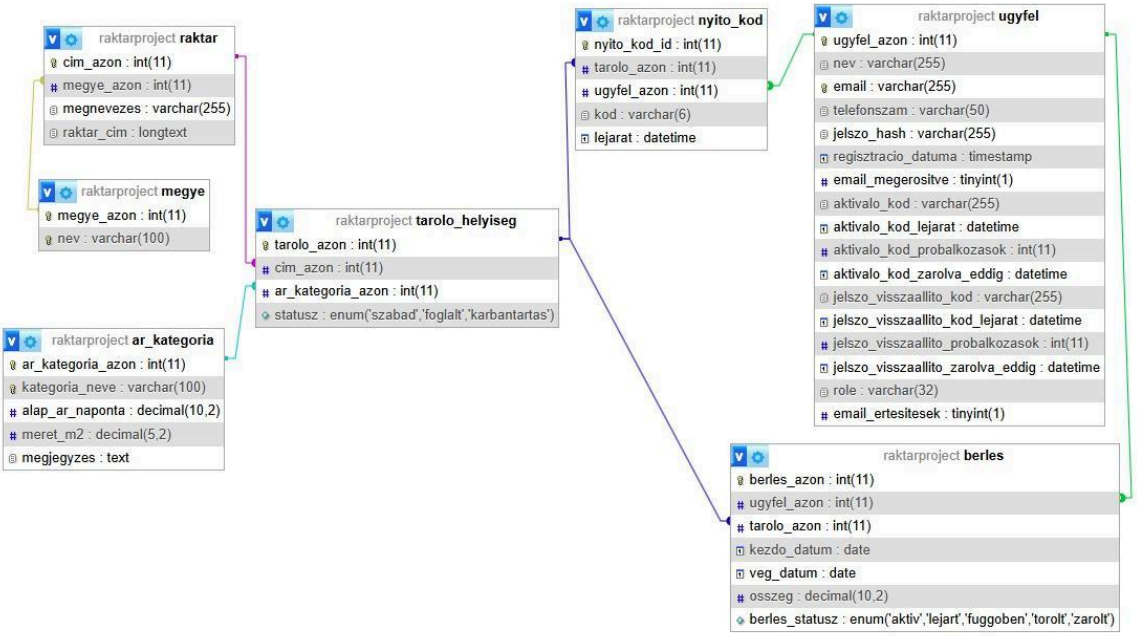
IT-Mentor. (2026.01.30). *Git verziókezelés alapjai és haladó technikák*. https://it-mentor.hu

Codecool Hungary. (2026.02.22). *Fullstack webfejlesztés tananyag*. https://codecool.com/hu

Mellékletek:

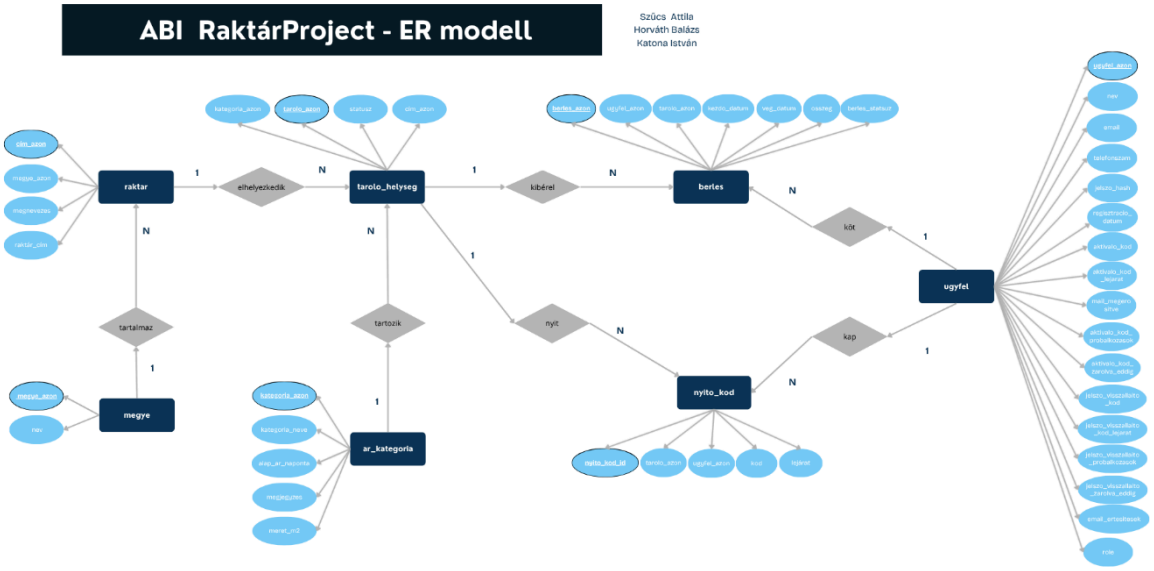
A mellékletekben kerülnek elhelyezésre az 1/3 oldalnál nagyobb ábrák, táblázatok, részletes telepítési leírás, forráskód részletek stb.

1. melléklet – Adatbázis séma:



16. sz. kép: Adatbázis séma Forrás: saját készítés.

2. melléklet – Projekt struktúra ER modell:

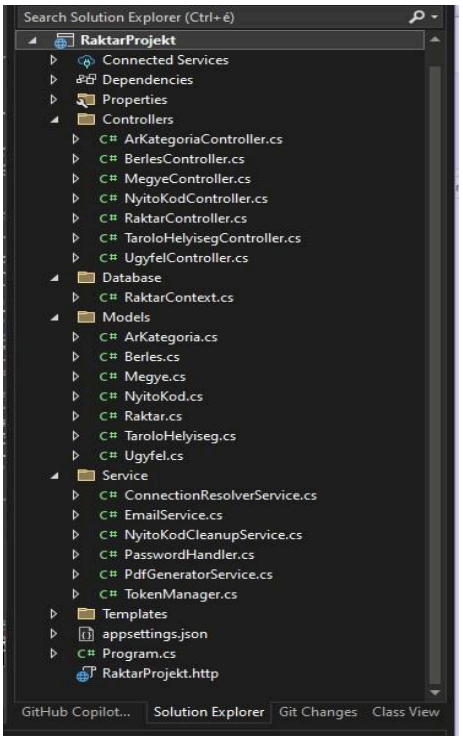


17. sz. kép: Er-modell Forrás: saját készítés.

```
namespace RaktarProjekt.Database
{
    18 references
    public class RaktarContext : DbContext
    {
        0 references
        public RaktarContext(DbContextOptions<RaktarContext> options) : base(options)
        {
        }
        6 references
        public DbSet<ArKategoria> ArKategoriak { get; set; }
        5 references
        public DbSet<Megye> Megyek { get; set; }
        7 references
        public DbSet<Raktar> Raktarak { get; set; }
        14 references
        public DbSet<TaroloHelyiseg> TaroloHelyisegek { get; set; }
        23 references
        public DbSet<Ugyfel> Ugyfelek { get; set; }
        22 references
        public DbSet<Berles> Berlesek { get; set; }
        12 references
        public DbSet<NyitoKod> NyitoKodok { get; set; }

        0 references
```

18. sz. kép: DbContext Forrás: saját készítés.



19. sz. kép: Backend mappa struktúra Forrás: saját készítés.

3. melléklet – API dokumentáció:

UgyfelController (/api/Ugyfel)		
Metódus	**Végpont**	**Megj.**
GET	/api/Ugyfel	*admin*
GET	/api/Ugyfel/{id}	
POST	/api/Ugyfel	*regisztráció*
POST	/api/Ugyfel/login	
POST	/api/Ugyfel/megerosites	
GET	/api/Ugyfel/megerosites/link	
POST	/api/Ugyfel/megerosites/ujrakuldes	
POST	/api/Ugyfel/jelszo-visszaallitas/keres	
POST	/api/Ugyfel/jelszo-visszaallitas/megerosites	
PUT	/api/Ugyfel/{id}/profil	
PUT	/api/Ugyfel/{id}/ertesitesek	
PUT	/api/Ugyfel/{id}/jelszo	
PUT	/api/Ugyfel/{id}/role	*admin*
GET	/api/Ugyfel/admin-check	*admin*
DELETE	/api/Ugyfel/{id}	*admin*

DELETE	/api/Ugyfel/{id:int}/torles	
BerlesController (/api/Berles)		
Metódus	**Végpont**	**Megj.**
GET	/api/Berles	
GET	/api/Berles/idointervallumok	
GET	/api/Berles/elerhetoseg	
GET	/api/Berles/sajat	*auth*
POST	/api/Berles	*auth*
POST	/api/Berles/fizetes	*auth*
PUT	/api/Berles/{id}/lezaras	*admin*
POST	/api/Berles/admin	*admin*
DELETE	/api/Berles/admin/{id}	*admin*
DELETE	/api/Berles/{id:int}	*auth*
TaroloHelyisegController (/api/TaroloHelyiseg)		
Metódus	**Végpont**	**Megj.**
GET	/api/TaroloHelyiseg	
GET	/api/TaroloHelyiseg/{id}	
POST	/api/TaroloHelyiseg	
PUT	/api/TaroloHelyiseg/{id}	
DELETE	/api/TaroloHelyiseg/{id}	
GET	/api/TaroloHelyiseg/admin/reszletes	*admin*
PUT	/api/TaroloHelyiseg/{id}/karbantartas	
RaktarController (/api/Raktar)		
Metódus	**Végpont**	**Megj.**
GET	/api/Raktar	
GET	/api/Raktar/{id}	
POST	/api/Raktar	
PUT	/api/Raktar/{id}	
DELETE	/api/Raktar/{id}	
MegyeController (/api/Megye)		
Metódus	**Végpont**	**Megj.**
GET	/api/Megye	
GET	/api/Megye/{id}	
POST	/api/Megye	
DELETE	/api/Megye/{id}	
ArKategoriaController (/api/ArKategoria)		
Metódus	**Végpont**	**Megj.**
GET	/api/ArKategoria	
GET	/api/ArKategoria/{id}	
POST	/api/ArKategoria	
PUT	/api/ArKategoria/{id}	
DELETE	/api/ArKategoria/{id}	
NyitoKodController		
Metódus	**Végpont**	**Megj.**

POST	/api/Nyitas/igenyles	*auth*
POST	/api/NyitoKod	*auth*
GET	/api/Nyitas/ellenorzes	
GET	/api/NyitoKod/aktiv	*auth*
DELETE	/api/NyitoKod/{taroloId}	*auth*
TavnyitasController (/api/Tavnyitas)		
Metódus	**Végpont**	**Megj.**
POST	/api/Tavnyitas/nyitas	*auth*
GET	/api/Tavnyitas/eszkoz-parancs	

A RaktárBérlés REST API-ja nyolc hauptController-en keresztül működik. Az összes végpont Bearer token autentikáció alapján működik (kivéve a regisztrációt és email verifikációt).